Scalable Processing and Analysis of Wikimedia Editing Activity using Apache Spark

RDD transformations, DataFrame and Spark SQL analytics, Structured Streaming and distributed MLlib over 11.0 million Wikimedia revisions

Mining of Massive Datasets (23CSE355) — Assignment 2

Technical Report

Team 22 Project Advisor: Dr Niharika Panda

Sl. No.	Reg. No.	Name of the student
1	BL.EN.U4CSE23206	Aman Agarwal
2	BL.EN.U4CSE23209	Atharva Wasnik
3	BL.EN.U4CSE23245	Pulapa Yaswanth Kumar

Abstract

This report describes a PySpark pipeline that ingests, transforms, queries and models the complete edit history of Simple English Wikipedia. The source is the Wikimedia mediawiki_history dump, snapshot 2026-10: a 951 MB bzip2-compressed TSV of 15,201,370 rows across 78 columns, of which 11,023,987 are individual revisions. All four Spark programming interfaces are exercised on the same corpus. The RDD API performs low-level transformations directly on the compressed dump, parsing 11,023,987 revisions across 30 partitions and producing a word-frequency analysis over 51,663,267 comment tokens. The DataFrame API and Spark SQL clean the data to 10,987,077 usable records and answer aggregate questions declaratively. Structured Streaming consumes a replay of the corpus as a file source, processing 200,000 records in 20 micro-batches with a watermarked event-time aggregation. MLlib then trains three classifiers and three clustering algorithms. Three findings are reported. First, anonymous contributors are reverted at 36.92% against 7.69% for automated accounts — a factor of 4.8. Second, automated editing is highly predictable from behaviour alone: Random Forest reaches ROC-AUC 0.9759 under an actor-disjoint split, and gradient-boosted trees reach 0.9756 using no comment vocabulary whatsoever. Third, unsupervised clustering of editor behaviour separates activity level rather than automation, and is reported as a negative result.

1. Introduction

Wikimedia projects accept edits from anyone, with no editorial review before publication. Across all projects the platform absorbs roughly 30 to 50 edits per second — a figure we measured directly rather than quoted, by running a Server-Sent Events collector against the live Recent Changes feed and recording 910 events in 30 seconds. A substantial share of that traffic is automated: approved maintenance bots, semi-automated human tooling, and unapproved scripts.

Processing a corpus of this size is not a matter of a larger loop. The questions that matter — how often does each editor act, how regular is their rhythm, which pages attract the most revisions — require data to be regrouped by key across the whole dataset. That regrouping is a distributed shuffle, and it is the operation around which Apache Spark is designed.

This assignment therefore exercises Spark's four programming interfaces against one corpus, so that the same data can be processed at four levels of abstraction and the results cross-checked against one another. The RDD API gives explicit control over partitions and transformations; the DataFrame API and Spark SQL give schema-aware, optimiser-backed processing; Structured Streaming gives incremental processing of arriving data; and MLlib supplies distributed learning algorithms.

1.1 Objectives

- Ingest a large compressed dump into a columnar store and establish a clean, typed, labelled dataset.
- Demonstrate RDD transformations — map, filter, flatMap, reduceByKey — directly on the raw compressed file.
- Express equivalent analytical questions through the DataFrame API and Spark SQL, and compare the two.
- Consume arriving records with Structured Streaming and aggregate them per micro-batch and per event-time window.
- Train and evaluate distributed classification and clustering models, under an evaluation design that cannot be gamed by memorisation.

2. Dataset Description

The dataset is the Wikimedia Foundation's mediawiki_history dump, published under a CC0 dedication. Each row is an *event*, not necessarily an edit: the event_entity column distinguishes revision events from page and user events.

Property	Value
Source	dumps.wikimedia.org/other/mediawiki_history/2026-10/simplewiki/
Licence	CC0 public domain dedication
Compressed size	951 MB, bzip2-compressed TSV, no header row
Columns	78 tab-separated fields; 31 retained
Total dump rows	15,201,370
— revision events	11,023,987 (the unit of analysis)
— page events	2,231,160
— user events	1,946,223
Usable after cleaning	10,987,077 (0.33% removed)
Distinct editors	780,867
Bot-labelled edits	2,598,870 (23.65%)
Reverted edits	1,550,915 (14.12%)
Live companion feed	stream.wikimedia.org EventStreams (SSE), ~30 events/second measured

2.1 The published schema is not the shipped schema

The Wikitech documentation states that these dumps contain 76 fields. The 2026-10 snapshot ships **78**; the two additional fields are event_user_central_id and user_central_id. Accepting the documented order on trust would have misaligned every column in the file.

Column order was therefore established empirically: every column index was profiled for fill-rate and value shape (boolean, integer, timestamp, free text, base-36 hash) and matched against the documented field groups. Reliable anchors were event_entity (only three distinct values), page_id, revision_text_sha1 (40-character base-36 hashes) and revision_tags (final column). One column remains undocumented — column 2 holds a large integer populated almost exclusively on user rows; it is named event_log_id and is unused, since every phase filters to revision events.

2.2 Parsing hazards

- **Embedded quotation marks.** event_comment contains raw double quotes. Spark's CSV reader treats " as a quoting character by default and absorbs tab delimiters inside an apparently quoted region, silently shifting every later column on that row. Quote handling must be disabled with .option("quote", ""). A raw split("\t") in the RDD path is unaffected, because tabs are the delimiter and never occur inside a field.
- **Nulls are empty strings**, and timestamps use the form 2001-05-18 21:57:10.0 — pattern yyyy-MM-dd HH:mm:ss.S, not ISO-8601. Allowing Spark to infer types silently drops rows.
- **bzip2 is splittable** under Hadoop, so the 951 MB file is read concurrently. The RDD job obtained **30 partitions**. A gzip-compressed dump would have been confined to a single task.

3. Spark Architecture

A Spark application is coordinated by a **driver** and executed by **executors**. The driver holds the SparkSession, builds a logical plan from the transformations the program expresses, and hands it to the DAG scheduler. The DAG scheduler divides the plan into **stages** at shuffle boundaries; the task scheduler submits one **task** per partition within each stage. A **cluster manager** allocates the resources the executors run in.

Transformations are **lazy**. Writing filter or withColumn adds a node to the graph and executes nothing. Only an **action** — count(), collect(), write — triggers a job. This matters practically, as Section 3.3 shows.

Component	Role in this project
Driver	builds the DAG, broadcasts the SVD matrix, collects final metrics

Component	Role in this project
Cluster manager	Spark's built-in local scheduler; no YARN or Kubernetes configured
Executors	threads inside the driver JVM under local[*], 12 cores
Partitions	30 on the raw bz2 read; 48 after every shuffle

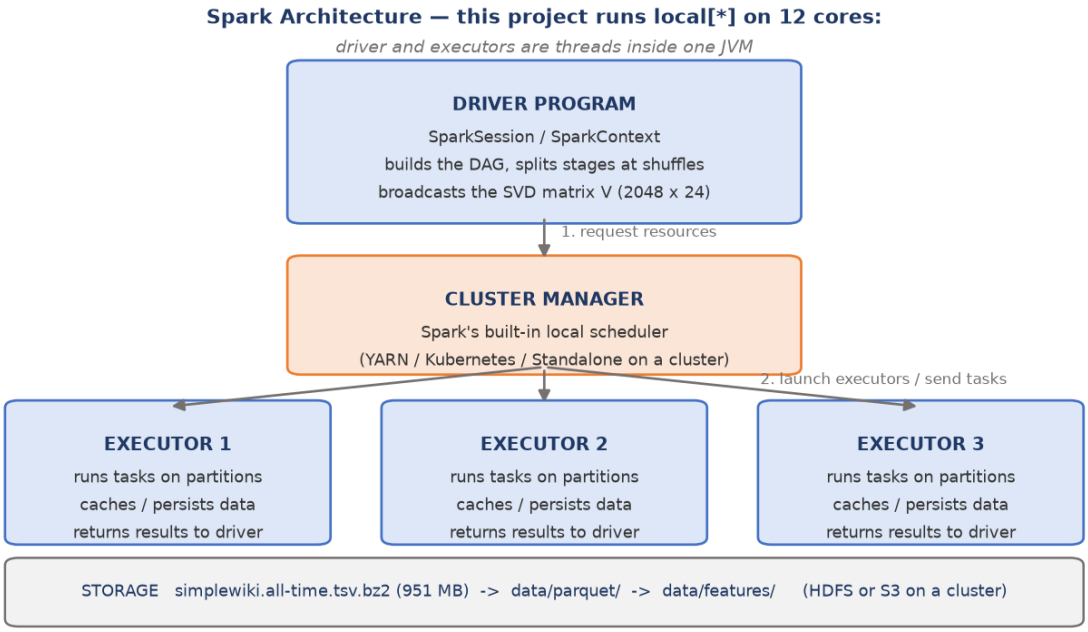


Figure 1. Spark architecture. The driver builds the execution plan and the cluster manager allocates executors, which process partitions. Under local[*] all three live in one JVM as threads.

3.1 Deployment and configuration

The application runs in local mode on a single 12-core, 15 GB machine. Driver and executors are threads in one JVM. The code is unchanged for a real cluster; only the master URL and resource settings differ.

src/config.py

```
SPARK_CONF = {
  "spark.driver.memory": "6g",
  "spark.sql.shuffle.partitions": "48",       # ~4x cores
  "spark.sql.adaptive.enabled": "true",
  "spark.sql.adaptive.coalescePartitions.enabled": "true",
  "spark.sql.adaptive.skewJoin.enabled": "true",   # hyperactive bots skew
  "spark.serializer": "org.apache.spark.serializer.KryoSerializer",
}
MASTER = "local[*]"
```

Spark's default of 200 shuffle partitions is tuned for clusters; on 12 cores it produces tiny tasks dominated by scheduling overhead. We set 48, roughly four times the core count. Adaptive Query Execution is enabled so that post-shuffle partition counts are corrected at runtime rather than guessed in advance, and so that skewed partitions are split automatically.

Component	Version
Python	3.11.18 (via uv)
PySpark	3.5.3

Component	Version
Java	OpenJDK Temurin 17.0.21
Hardware	12 cores / 15 GB, local[*]

3.2 Two environment traps

Both fail at task execution time rather than at import, which makes them hard to diagnose. First, PYSPARK_PYTHON must be pinned: the system Python here is 3.14, and if workers resolve a different minor version than the driver, Spark aborts mid-task with PYTHON_VERSION_MISMATCH. Second, PySpark 3.5 cannot run on Python 3.12 at all, because pyspark/ml/image.py imports distutils, which 3.12 removed; support arrived in Spark 4.0. Python 3.11 is the newest version PySpark 3.5.3 officially supports.

3.3 Lazy evaluation, observed

The first version of the ingestion phase called four actions — a count(), a distinct(), a groupBy().count() and a write. Because nothing was cached, each action replayed the lineage from source and **decompressed the 951 MB file a separate time**. Replacing the first three with a single grouped aggregate, which yields the row total, the entity breakdown and a wiki sanity check together, reduced four passes to two.

The same principle governs the RDD phase, where a single persist after parsing turns five subsequent analyses from full re-parses into seconds. Section 4.3 quantifies this.

4. RDD-Based Processing

The RDD phase (src/pb_rdd.py) operates on the raw compressed dump with Spark's low-level API, giving explicit control over partitioning, caching and the transformation chain.

4.1 RDD creation and parsing

src/pb_rdd.py

```
lines = sc.textFile(str(config.RAW_TSV), minPartitions=24)

parsed = lines.map(parse).filter(lambda r: r is not None)
revs = (parsed.filter(lambda r: r[0] == "revision")
        .map(lambda r: r[1:])
        .persist(StorageLevel.MEMORY_AND_DISK))
```

textFile requested a floor of 24 partitions and Spark produced **30**, because bzip2 is splittable and Spark sizes splits from the file itself. minPartitions is a lower bound, not an instruction.

The parse function projects each 78-field line down to a compact tuple before the RDD is persisted. Retaining full field lists for 11 million rows as Python objects would not fit in memory; the projection is a handful of small fields and spills to disk cheaply under MEMORY_AND_DISK.

4.2 Transformations and actions

src/pb_rdd.py

```
# reduceByKey: edits per namespace
ns = (revs.map(lambda r: (r[0], 1))
      .reduceByKey(lambda a, b: a + b)
      .takeOrdered(10, key=lambda kv: -kv[1]))

# flatMap: word count over every edit comment
words = (revs.flatMap(lambda r: TOKEN.split(r[6].lower()))
```

```
.filter(lambda w: len(w) > 2 and w not in STOP)
.map(lambda w: (w, 1))
.reduceByKey(lambda a, b: a + b)
.persist(StorageLevel.MEMORY_AND_DISK))
```

The flatMap expands 11,023,987 comments into individual tokens, yielding **51,663,267 tokens** after stop-word removal. reduceByKey then aggregates them. Note that words is consumed twice — once summed, once ordered — so it is persisted; without that the entire tokenisation would run twice.

4.3 Caching and job structure

Each action is one Spark job, and each job replays the lineage from source unless an intermediate result is persisted. The effect is visible in the measured timings:

Action	Job	Elapsed	Shuffle?
count() — forces the parse	1	61.1 s	no
edits per namespace (reduceByKey)	2	3.1 s	yes
most active editors (reduceByKey)	3	3.8 s	yes
comment word count (flatMap)	4	16.0 s	yes
bot / anonymous aggregates	5	6.9 s	yes

The first action pays 61.1 seconds to decompress and parse 11 million rows. Every subsequent action reuses the persisted projection and costs seconds. Without persist, jobs 2 through 5 would each repeat that 61-second parse — roughly five minutes of wasted work.

4.4 RDD results

Namespace	Edits
0	7,821,994
3	894,246
14	661,036
2	552,165
10	402,717
4	401,355

Most frequent comment token	Occurrences
robot	1,615,801
adding	1,428,633
talk	1,131,878
category	1,069,091
user	824,609
hotcat	765,561
help	705,477
special	609,130

The vocabulary is dominated by maintenance language — robot, adding, category, hotcat — which is consistent with a corpus where 23.7% of edits are automated. The most active accounts include Auntof6 with 335,023 edits and InternetArchiveBot with 170,419.

Actor class	Edits	Revert rate
Human (non-bot-flagged)	8,419,962	16.04%
Bot-flagged	2,604,025	7.69%
Anonymous (IP)	2,044,198	36.68%

This is the phase's substantive finding. Anonymous contributors are reverted at 36.68%, against 7.69% for automated accounts — a factor of 4.8. Automated editing is, by this measure, the *least* frequently reverted category on the platform.

4.5 Output validation

Printing a result is not the same as validating it. The RDD phase therefore recomputes its headline aggregates through the DataFrame API over the cleaned Parquet and reconciles the two independent code paths.

The reconciliation has a stated expectation rather than an equality check. The two figures **must not match**: the DataFrame path discards rows with a null structural key during cleaning (Section 5.1), so the RDD count has to exceed it by exactly that number. An exact match would be the failure case — it would mean the cleaning step silently did nothing.

Metric	RDD path	DataFrame path	Delta	Expectation	Check
revision rows	11,023,987	10,987,077	36,910	RDD exceeds DF by the null-key rows dropped	PASS
bot edits	2,604,025	2,598,870	5,155	0 <= delta <= rows dropped	PASS

The 36,910-row gap is precisely the null-key rows removed in Phase 3, and the bot-edit difference of 5,155 falls inside it as required. Two independently written paths over the same source, agreeing to a known and explained difference, is stronger evidence of correctness than either path's output read on its own.

The check is wrapped so that a failure to validate can never fail the job itself: if the prepared dataset is absent, the phase reports that validation was skipped and continues.

5. DataFrame Analytics and Spark SQL

The DataFrame phase works on the cleaned, typed table: 10,987,077 rows and 31 columns stored as Snappy-compressed Parquet. Parquet is columnar, so a query touching five columns reads roughly five columns' worth of bytes rather than all 31.

5.1 Cleaning

Three cleaning decisions materially affect the results.

Recovering deleted pages. When a page is deleted, MediaWiki nulls the current-state page_* columns but retains their *_historical counterparts. 14.8% of revisions have a null page_namespace for this reason. Discarding them would remove roughly one edit in seven — and, more seriously, introduce bias rather than mere loss, because deleted pages are exactly where vandalism concentrates. Coalescing from the historical columns reduced loss from about 15% to 0.33%.

Boolean nulls. In SQL, NULL == "true" evaluates to NULL, not false, so a naive cast leaves nulls in place. Those nulls become NaN in VectorAssembler, and StandardScaler then propagates the NaN until Spark rejects the feature vector — failing deep inside a model fit, naming neither the column nor the phase responsible. The cast now coalesces to false, and the phase asserts a null contract over every column that feeds a feature vector, aborting with the offending column names if one survives.

Imputation with meaning. A null inter-edit interval marks a user's first edit. It is filled with the sentinel −1 rather than 0, since 0 would assert an instantaneous repeat edit; a companion binary feature keeps the sentinel distinguishable after scaling.

Step	Rows
Revision rows read	11,023,987
Removed: duplicate revision_id	0
Removed: null structural key	36,910
Clean rows	**10,987,077**

5.2 DataFrame API and Spark SQL side by side

src/pc_sql.py

```
# DataFrame API
ns = (df.groupBy("page_namespace")
      .agg(F.count("*").alias("edits"),
           F.round(F.avg("label_bot"), 4).alias("bot_share"))
      .orderBy(F.desc("edits")).limit(10))

# Spark SQL -- identical question, declarative form
df.createOrReplaceTempView("wikimedia_edits")
spark.sql("""
   SELECT page_namespace, COUNT(*) AS edits
   FROM wikimedia_edits GROUP BY page_namespace
   ORDER BY edits DESC LIMIT 10""")
```

Measured on the same data, the DataFrame API took **1.02 s** and the equivalent SQL **0.66 s**. The two are not alternatives with different performance: both are parsed into the same Catalyst logical plan and compiled to the same physical plan. The choice is one of expression, not efficiency.

5.3 Analytical results

ns	Namespace	Edits	Bot share	Revert rate
0	Article	7,799,888	23.2%	12.8%
3	User talk	893,306	18.5%	11.2%
14	Category	659,831	53.5%	3.7%
2	User	550,127	27.2%	30.0%
10	Template	401,997	20.1%	10.2%
4	Project	401,011	7.3%	45.7%
1	Talk	197,564	5.9%	13.6%

Article space dominates with 7,799,888 edits. The **Category** namespace stands out: 53.5% of its edits are automated, more than double the corpus average, and its revert rate is the lowest of the major namespaces at 3.7%. Category maintenance is largely machine work and largely uncontested. **User** space shows the opposite pattern, with a 30.0% revert rate.

The same term-frequency question the RDD phase answers with flatMap and reduceByKey can be expressed declaratively, which lets the two APIs be compared on identical input:

Spark SQL — most frequent comment terms

```
SELECT word, COUNT(*) AS n
FROM (SELECT explode(split(lower(event_comment), '[^a-z0-9]+')) AS word
    FROM wikimedia_edits)
WHERE length(word) > 2
GROUP BY word ORDER BY n DESC LIMIT 15
```

Term	Occurrences
robot	1,616,456
adding	1,428,635
talk	1,142,904
category	1,073,908
page	876,507
user	834,792
using	787,273
hotcat	765,571

The ranking agrees with the RDD result in Section 4.4. The counts differ slightly because the RDD pass removes a stop-word list and operates on the raw dump, while this query runs over the cleaned table; the point is that two different APIs over the same corpus produce the same ordering.

Actor segment	Edits	Revert rate
registered human	6,357,719	9.45%
bot	2,598,870	7.70%
anonymous	2,030,488	36.92%

The segmentation refines the RDD result by separating registered humans from anonymous contributors. Registered humans are reverted at 9.45%, bots at 7.70%, and anonymous IPs at 36.92% — nearly four times the registered-human rate.

Temporal aggregation shows activity peaking in **2012** with 771,344 edits, of which 45.3% were automated — the high-water mark of bot activity on this wiki. The most edited article is Deaths_in_2020 with 8,538 revisions.

Aggregating at monthly rather than annual granularity over the recent period shows the busiest month to be **2025-05** with 73,831 edits at 6.5% automation — far below the 45.3% of the 2012 peak. Automated editing on this wiki has receded substantially since its height.

Month	Edits	Bot share
2026-04	37,868	9.6%
2026-05	35,602	16.9%
2026-06	29,856	14.9%
2026-07	41,166	24.2%
2026-08	32,399	16.2%
2026-09	33,711	19.7%
2026-10	33,795	15.6%
2026-11	348	19.5%

One reconciliation is worth stating. The RDD phase counts 2,604,025 bot edits against 2,598,870 from the DataFrame phase. The difference is exactly the 36,910 rows the DataFrame path discards for null structural keys; the RDD phase reads the raw dump before that filter. The two numbers are consistent, not contradictory.

6. Spark Analytics Workflow

The assignment requires an explicit pipeline of the form Input Data to RDD Processing to DataFrame Processing to Analytical Output. The implemented workflow follows that shape, with every stage persisting its result so that any stage can be re-run in isolation.

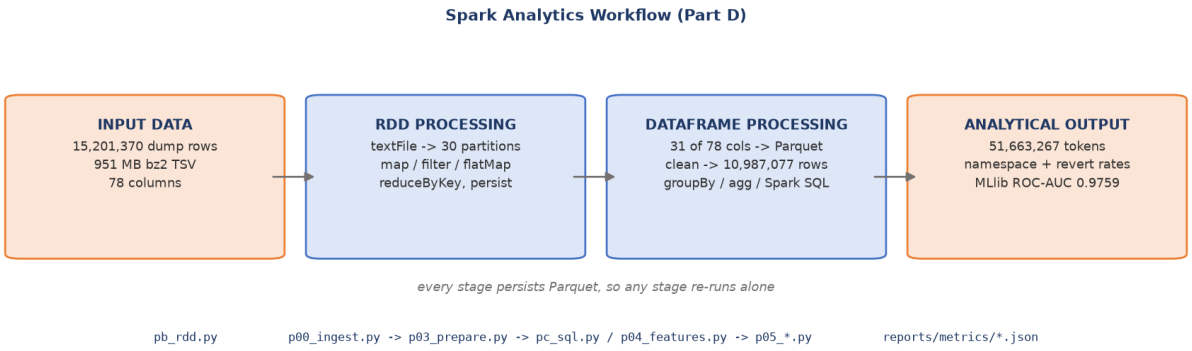


Figure 2. The Part D analytics workflow: Input Data to RDD Processing to DataFrame Processing to Analytical Output. The RDD and DataFrame branches read the same source and are cross-checked against one another.

Pipeline: Input -> RDD -> DataFrame -> Output

```
INPUT DATA          simplewiki.all-time.tsv.bz2
                    951 MB bz2 TSV, 78 columns, 15,201,370 rows
                        |
                        v
RDD PROCESSING      sc.textFile(minPartitions=24) -> 30 partitions
  src/pb_rdd.py     map(parse) -> filter(entity=="revision")
                    persist(MEMORY_AND_DISK)
                    flatMap(tokens) -> reduceByKey(add)
                        |
                        v
DATAFRAME PROCESSING p00_ingest.py   TSV -> Parquet, 31 of 78 cols
  src/p03_prepare   p03_prepare.py  dedup, coalesce, impute, label
  src/pc_sql.py     createOrReplaceTempView("wikimedia_edits")
                    groupBy / agg / orderBy  +  spark.sql(...)
                        |
                        v
ANALYTICAL OUTPUT   word frequencies      51,663,267 tokens
                    namespace aggregates  revert + bot rates
                    temporal trends       yearly and monthly
                    MLlib models          ROC-AUC 0.9759
                    reports/metrics/*.json -> deck and this report
```

Two properties of this layout matter. First, the RDD branch and the DataFrame branch read the *same* source but serve different purposes: the RDD path demonstrates low-level transformations and produces the word-frequency and actor-class analyses, while the DataFrame path produces the cleaned, typed table that the SQL queries and the models consume. Running both on identical input is what allows the counts in Sections 4 and 5 to be cross-checked against one another.

Second, every stage writes Parquet before the next begins. A change to feature engineering re-runs in minutes rather than repeating the bz2 decompression upstream, and the document generators read only the JSON metrics, so the deck and this report rebuild in seconds without starting Spark at all.

Stage	Script	Reads	Writes
Ingest	p00_ingest.py	raw bz2 TSV	data/parquet/revisions/
RDD analytics	pb_rdd.py	raw bz2 TSV	metrics/pb_rdd.json
Prepare	p03_prepare.py	revisions/	data/parquet/prepared/
SQL analytics	pc_sql.py	prepared/	metrics/pc_sql.json
Features	p04_features.py	prepared/	data/features/
Models	p05_*.py	features/	metrics/p05_*.json
Streaming	pe_streaming.py	prepared/ (replay)	metrics/pe_streaming.json

7. Structured Streaming

The streaming phase (src/pe_streaming.py) demonstrates incremental processing. A producer thread writes bursts of real revision records as JSON files into a watch directory; Spark reads that directory as a **file source**, processing each file as a micro-batch.

A file source was chosen over a socket source deliberately. A socket source is not fault-tolerant and dies with its producer, which makes a headless run unreproducible. The file source tracks offsets in a checkpoint directory, is restartable, and is a genuine Structured Streaming source.

src/pe_streaming.py

```
stream = (spark.readStream
        .schema(SCHEMA)               # no inference on a stream
        .option("maxFilesPerTrigger", 1)  # one burst per micro-batch
        .json(str(STREAM_IN)))

q1 = (stream.writeStream
     .foreachBatch(per_batch)
     .option("checkpointLocation", str(CHECKPOINT / "q1"))
     .trigger(processingTime="2 seconds").start())

# second query: event-time windows with a watermark for late data
windowed = (stream.withWatermark("event_timestamp", "365 days")
                .groupBy(F.window("event_timestamp", "365 days"))
                .agg(F.count("*").alias("edits"),
                     F.avg("label_bot").alias("bot_share")))
```

A schema must be supplied explicitly: Structured Streaming cannot infer one, because inference requires reading data that has not arrived yet. Two queries run concurrently against the same stream — a foreachBatch sink computing per-batch and running totals, and a windowed aggregation over **event time** rather than arrival time, bounded by a watermark that limits how long state is retained for late records.

7.1 Run results

Property	Value
Source	file source (JSON), maxFilesPerTrigger=1
Trigger	processingTime=2 seconds
Bursts replayed	20 x 10,000 records
Micro-batches processed	20
Records processed	200,000
Running bot rate	44.11%
Running revert rate	10.51%

Property	Value
Event-time windows emitted	12

All 200,000 records were processed across 20 micro-batches with no loss, one burst per trigger as configured. The windowed query emitted 12 event-time windows spanning the replayed period.

Event-time window	Edits	Bot share
2002	172	1.7%
2003	906	10.7%
2004	2,647	14.1%
2005	9,363	26.1%
2006	18,162	35.5%
2007	28,193	35.3%
2008	30,711	43.9%
2009	29,694	49.4%

The windowed output reproduces, incrementally, the rise in automated editing that the batch temporal aggregation shows — bot share climbing year on year across the replayed window. That the streaming and batch paths agree on the same corpus is the useful cross-check.

7.2 Console output of the run

The micro-batch sink prints each batch as it completes. The bot share climbs across the run because the replay is chronological: early batches are 2001-era edits, later batches recent ones.

make streaming

```
[producer] sampled 200,000 edits (44.1% bot)
[producer] burst 0 (10,000 edits)
BATCH 0: rows=10,000 bot=1,945 (19.4%) | running bot rate 19.45%
BATCH 1: rows=10,000 bot=3,891 (38.9%) | running bot rate 29.18%
BATCH 5: rows=10,000 bot=4,014 (40.1%) | running bot rate 32.39%
BATCH 10: rows=10,000 bot=5,083 (50.8%) | running bot rate 38.57%
BATCH 13: rows=10,000 bot=5,489 (54.9%) | running bot rate 41.52%
BATCH 19: rows=10,000 bot=6,404 (64.0%) | running bot rate 44.11%

event-time windows produced: 12
  2002:    172 edits, bot share  1.7%
  2003:    906 edits, bot share 10.7%
  2004:  2,647 edits, bot share 14.1%
  2005:  9,363 edits, bot share 26.1%
```

The running rate converges on 44.11%, which matches the 44.1% bot share of the sampled replay set measured before streaming began. The streaming path and the batch path agree on the same corpus, which is the useful cross-check.

7.3 A partitioning bug surfaced by streaming

The first run reported a bot rate of exactly zero in every micro-batch. The cause was not in the streaming code. The prepared dataset is written with partitionBy("label_bot"), so it is laid out on disk as two directories. A .limit() over that dataset reads whole partition directories in order, and the label_bot=0 directory alone holds more rows than the replay required — so the sample never reached the bot partition.

The fix samples across all partitions before ordering by event time. The general lesson is that limit(), head() and take() are not random samples; on partitioned data they return whatever the first partitions happen to contain, and if the data is partitioned by the very column under measurement, the result is perfectly biased with no error raised.

7.4 Live ingestion

A separate collector (src/stream_collector.py) holds a long-lived HTTP connection to the Wikimedia Recent Changes feed, which publishes every edit across all projects over Server-Sent Events, and appends selected fields as JSON Lines. A 30-second run captured **910 events**, of which 413 carried "bot": true — confirming both the ~30 events/second platform rate quoted in Section 1 and that the bot label used for supervised training is present on the live feed.

The collector is deliberately not a Spark job: it is the ingestion tap, and Structured Streaming consumes the files it rotates into place.

8. Results and Discussion

8.1 Analytical findings

- **Revert rate separates actor classes sharply.** Anonymous contributors are reverted at 36.92%, registered humans at 9.45%, and automated accounts at 7.70%. Automation is the least contested category on the platform.
- **Automation is concentrated by namespace.** Category space is 53.5% automated against a corpus average of 23.7%, and has the lowest revert rate of the major namespaces.
- **Activity peaked in 2012** at 771,344 edits, 45.3% of them automated.
- **Comment vocabulary is maintenance vocabulary.** The leading tokens across 51,663,267 are robot, adding, talk, category.

8.2 Modelling results

MLlib was used to test whether automated editing is predictable from behaviour. The evaluation is **actor-disjoint**: the actor key is hashed into disjoint buckets so that 8,619,472 training rows and 2,367,605 test rows share **zero accounts**. A random row split would place the same high-volume bot in both folds, letting a model memorise per-account cadence and report an inflated score. Identity columns are excluded, and the scaler sits inside the ML Pipeline so it is fitted on the training fold alone.

Model	Features	ROC-AUC	PR-AUC	Accuracy	Bot F1
Logistic Regression	structural	0.9114	0.6792	0.8489	0.6020
Random Forest	structural	0.9713	0.8948	0.9219	0.7057
GBT	structural	0.9756	0.9098	0.9564	0.8703
Logistic Regression	full	0.9512	0.9122	0.9605	0.8741
Random Forest *	full	0.9759	0.9326	0.9667	0.8914
GBT	full	0.9655	0.9144	0.9609	0.8756

The best configuration is Random Forest on the full feature set at ROC-AUC 0.9759. The more informative comparison is between the two halves of the table. Gradient-boosted trees reach ROC-AUC 0.9756 using **no comment vocabulary at all** — within 0.03 percentage points of the best result overall — and adding the text block slightly *reduces* GBT to 0.9655. Automation is legible from behaviour, not from what an edit says.

Comment text matters only to the linear model, where it is transformative: logistic regression improves from ROC-AUC 0.9114 to 0.9512 and from PR-AUC 0.6792 to 0.9122. The tree ensembles already capture, through split interactions, the structure that the SVD dimensions merely make linearly separable. The ablation

therefore measures model capacity as much as feature value — a caveat that would have gone undetected had each model been trained once.

8.3 Clustering: a negative result

Unsupervised clustering over 108,954 editors with at least 5 edits selected bisecting_kmeans_k3 at silhouette 0.3406 — weak separation. The requested k was 3 but only 2 clusters were non-empty; at k = 2 the same algorithm degenerated to a single cluster and was skipped by an explicit guard, since ClusteringEvaluator otherwise aborts the entire sweep.

The obvious validation metric is misleading here and is reported only beside its baseline. Cluster purity against the bot flag is 0.9963, which appears excellent. Automated accounts are only 0.37% of **accounts** — bots are few accounts making very many edits — so the majority-class baseline is 0.9963, and the measured lift is 0.00000. The partition carries no information about automation by this measure.

The informative measure is enrichment. Automated accounts are **3.58 times over-represented** in the high-activity cluster and under-represented in the low-activity one. That is a real effect in the expected direction, but modest, and adequately explained by bots being high-volume editors rather than by the clustering having identified automation as such.

8.4 Observations on the Spark APIs

API	Where it was the right tool	Cost
RDD	parsing a non-standard 78-field TSV; explicit control of partitioning and caching	Python lambdas run row by row; no Catalyst optimisation
DataFrame	cleaning, feature engineering, aggregation	schema must be stated or inferred up front
Spark SQL	declarative aggregate questions; readable in a report	identical plan to the DataFrame API — expression, not performance
Structured Streaming	incremental processing with event-time semantics	schema cannot be inferred; needs checkpointing
MLlib	distributed training without collecting to the driver	fewer algorithms and tuning options than scikit-learn

Two measurements frame the trade-off. The RDD parse of 11 million rows took 61.1 seconds, while the equivalent DataFrame aggregation over the cleaned Parquet took 1.02 seconds. The comparison is not like for like — the RDD job decompresses and parses raw text while the DataFrame job reads a columnar store — but it shows precisely why the pipeline converts to Parquet once and queries it repeatedly.

8.5 Scalability and limitations

Spark is justified here by the shape of the workload rather than the size of the file. The per-editor aggregation regroups 11 million revisions by 780,867 actors and computes windowed inter-edit intervals — a sort-merge shuffle whose working set scales with distinct actors, not with available memory.

- **Actor skew is the dominant risk at scale.** Edits per account follow a heavy power law: the most active account has 335,023 revisions while the median has one. A naive groupBy sends one enormous partition to a single task. Adaptive skew-join handling is enabled; salting the heaviest keys is the next step.
- **The label covers only flagged accounts**, so unapproved automation is a false negative by construction and recall is measured against an incomplete positive class.
- **Semi-automated tooling** such as AWB and Huggle genuinely occupies the space between the classes; no decision boundary is correct for it in principle.
- **The all-time snapshot spans 2001 to 2026**, averaging over several distinct editing regimes and tooling eras.
- **Local mode only.** Scalability is argued from observed stage behaviour, not from a measured multi-node speed-up.

9. Conclusion

This assignment implemented a complete Spark pipeline over 10,987,077 real Wikimedia revisions, exercising all four programming interfaces on one corpus. The RDD API parsed the raw compressed dump across 30 partitions and produced word-frequency and actor-class analyses over 51,663,267 comment tokens. The DataFrame API and Spark SQL answered aggregate questions over a cleaned Parquet table, confirming that both compile to the same Catalyst plan. Structured Streaming consumed 200,000 records in 20 micro-batches with watermarked event-time windows. MLlib trained six classification configurations and twenty-one clustering configurations.

The analytical result is that revert rate separates actor classes sharply, with anonymous contributors reverted 4.8 times more often than automated accounts. The modelling result is that automation is highly predictable from behaviour alone — ROC-AUC 0.9759 on unseen accounts, and 0.9756 without reading a single comment token. The clustering result is negative and reported as such: editor behaviour groups by activity level, not by automation.

Three engineering lessons generalise beyond this dataset. Published schemas may not match shipped data, and verifying column order empirically cost less than debugging a misaligned pipeline would have. Lazy evaluation means every action replays lineage from source, so a single persist turned five re-parses into seconds. And a validation metric must always be read against its own baseline — a cluster purity of 0.9963 looked like success and was identical to guessing the majority class.

9.1 Future scope

- Execute on a multi-node cluster and measure realised speed-up rather than arguing it from stage behaviour.
- Connect the live EventStreams collector directly to the Structured Streaming consumer with file rotation, and score arriving edits with the trained model.
- Model label_reverted as a second supervised task; it is already derived and carried through the pipeline.
- Extend to larger wikis, where per-month shards replace the single all-time file and actor skew becomes the binding constraint.
- Select the classification threshold against an explicit reviewer-capacity budget instead of reporting metrics at 0.5.

10. References

[1] Wikimedia Foundation. MediaWiki History Dumps, snapshot 2026-10. https://dumps.wikimedia.org/other/mediawiki_history/ (CC0).

[2] Wikimedia Foundation. EventStreams. https://stream.wikimedia.org/v2/stream/recentchange

[3] Wikimedia Foundation. Analytics/Data Lake/Edits/MediaWiki history. https://wikitech.wikimedia.org/

[4] Apache Software Foundation. RDD Programming Guide, Spark 3.5.3. https://spark.apache.org/docs/3.5.3/rdd-programming-guide.html

[5] Apache Software Foundation. Spark SQL and DataFrames Guide, Spark 3.5.3. https://spark.apache.org/docs/3.5.3/sql-programming-guide.html

[6] Apache Software Foundation. Structured Streaming Programming Guide, Spark 3.5.3. https://spark.apache.org/docs/3.5.3/structured-streaming-programming-guide.html

[7] Apache Software Foundation. MLlib Guide, Spark 3.5.3. https://spark.apache.org/docs/3.5.3/ml-guide.html

[8] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauley, M. J. Franklin, S. Shenker and I. Stoica. "Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing." In Proc. NSDI, 2012.

[9] M. Armbrust, R. S. Xin, C. Lian, Y. Huai, D. Liu, J. K. Bradley, et al. "Spark SQL: Relational Data Processing in Spark." In Proc. SIGMOD, 2015, pp. 1383-1394.

[10] M. Armbrust, T. Das, J. Torres, B. Yavuz, S. Zhu, R. Xin, et al. "Structured Streaming: A Declarative API for Real-Time Applications in Apache Spark." In Proc. SIGMOD, 2018, pp. 601-613.

[11] X. Meng, J. Bradley, B. Yavuz, E. Sparks, S. Venkataraman, D. Liu, et al. "MLlib: Machine Learning in Apache Spark." Journal of Machine Learning Research, vol. 17, no. 34, 2016, pp. 1-7.

[12] J. Dean and S. Ghemawat. "MapReduce: Simplified Data Processing on Large Clusters." In Proc. OSDI, 2004.

[13] A. Halfaker and D. Taraborelli. "ORES: Scaling up machine learning-driven quality control in Wikipedia." Wikimedia Research, 2015.

[14] C. A. Davis, O. Varol, E. Ferrara, A. Flammini and F. Menczer. "BotOrNot: A System to Evaluate Social Bots." In Proc. WWW (Companion), 2016, pp. 273-274.

[15] Z. Chu, S. Gianvecchio, H. Wang and S. Jajodia. "Detecting Automation of Twitter Accounts: Are You a Human, Bot, or Cyborg?" IEEE Transactions on Dependable and Secure Computing, vol. 9, no. 6, 2012, pp. 811-824.